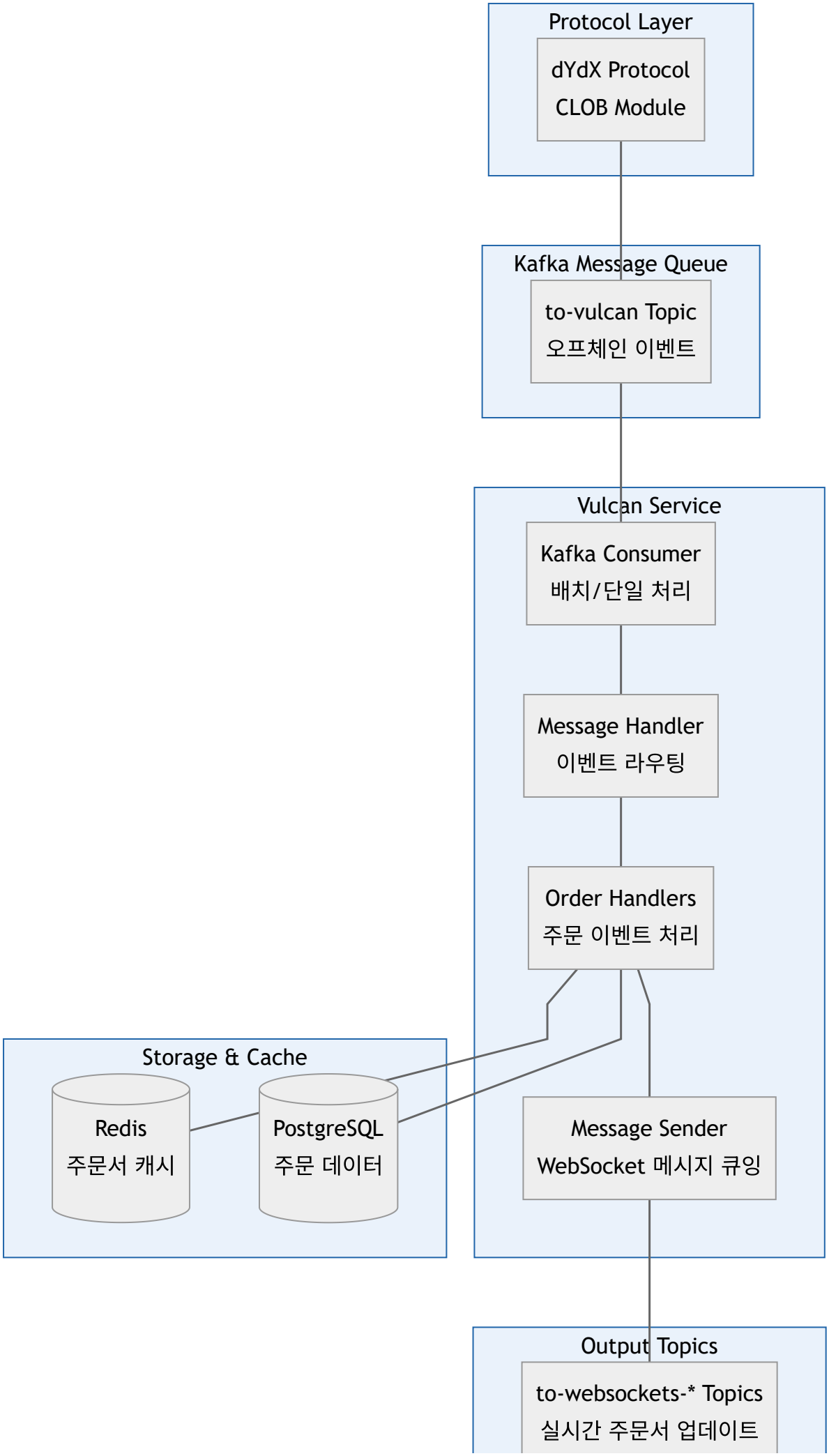
Vulcan 서비스 구현 분석

개요

Vulcan은 dYdX v4 Indexer의 오프체인 데이터 아카이브(Off-chain Data Archival) 서비스입니다. Protocol에서 발생하는 오프체인 이벤트들, 특히 주문서 관련 이벤트들(주문 생성, 업데이트, 취소)을 처리하여 Redis 캐시를 업데이트하고 실시간 WebSocket 메시지를 생성하는 역할을 담당합니다.

아키텍처 개요


Protocol Layer
dYdX Protocol
CLOB Module
Kafka Message Queue
to-vulcan Topic
오프체인 이벤트
Vulcan Service
Kafka Consumer
배치/단일 처리
Message Handler
이벤트 라우팅
Order Handlers
주문 이벤트 처리
Storage & Cache
Redis
주문서 캐시
PostgreSQL
주문 데이터
Message Sender
WebSocket 메시지 큐잉
Output Topics
to-websockets-* Topics
실시간 주문서 업데이트

오프체인 이벤트 처리 방식

1. 이벤트 수신 및 라우팅

Vulcan은 `to-vulcan` Kafka 토픽에서 `OffChainUpdateV1` 메시지를 수신합니다:

```
export async function onMessage(message: KafkaMessage): Promise<void> {
const messageValue: Buffer = message.value;
const update: OffChainUpdateV1 = getOffChainUpdate(messageValue, offset);
validateOffChainUpdate(update);
// 이벤트 타입에 따른 핸들러 선택
const handler: Handler = new (getHandler(update))!(
getTransactionHashFromHeaders(message.headers)
);
await handler.handleUpdate(update);
}

function getHandler(update: OffChainUpdateV1): HandlerInitializer | undefined {
if (update.orderUpdate !== undefined) {
return OrderUpdateHandler;
} else if (update.orderPlace !== undefined) {
return OrderPlaceHandler;
} else if (update.orderRemove !== undefined) {
```

```
return OrderRemoveHandler;
}
return undefined;
}
```

2. 배치 처리 시스템

Vulcan은 높은 처리량을 위해 배치 처리를 지원합니다:

```
export async function onBatch(payload: EachBatchPayload): Promise<void> {
const batch: Batch = payload.batch;
let lastCommitTime: number = Date.now();
for (let i = 0; i < batch.messages.length; i++) {
const message: KafkaMessage = batch.messages[i];
await onMessage(message);

// 주기적으로 오프셋 커밋하여 연결 유지
const now: number = Date.now();
if (now - lastCommitTime > config.KAFKA_BATCH_PROCESSING_COMMIT_FREQUENCY_MS) {
payload.resolveOffset(message.offset);
await Promise.all([
payload.heartbeat(),
payload.commitOffsetsIfNecessary(),
]);
```

```
lastCommitTime = now;

}

}

}
```

3. 이벤트 검증 시스템

모든 오프체인 업데이트는 엄격한 검증을 거칩니다:

```
function validateOffChainUpdate(update: OffChainUpdateV1) {

if (update.orderUpdate === undefined &&

update.orderPlace === undefined &&

update.orderRemove === undefined) {

throw new ParseMessageError('Message does not contain an order update, place,

}

}
```

주문서 업데이트 및 시장 데이터 처리 로직

1. 주문 생성 처리 (OrderPlaceHandler)

주문 생성 이벤트는 다음과 같이 처리됩니다:

```
export class OrderPlaceHandler extends Handler {

protected async handle(update: OffChainUpdateV1): Promise<void> {
```

```
const orderPlace: OrderPlaceV1 = update.orderPlace!;

const order: IndexerOrder = orderPlace.order!;

// 영구 선물 마켓 정보 조회

const perpetualMarket = perpetualMarketRefresher

.getPerpetualMarketFromClobPairId(order.orderId!.clobPairId.toString());

// Redis 주문 객체 생성

const redisOrder: RedisOrder = convertToRedisOrder(order, perpetualMarket);

// Redis 캐시에 주문 추가

const placeOrderResult: PlaceOrderResult = await placeOrder({

redisOrder,

client: redisClient,

});

// 취소된 주문 캐시에서 제거

await this.removeOrderFromCanceledOrdersCache(

OrderTable.orderIdToUuid(redisOrder.order?.orderId!)

);

// 가격 레벨 업데이트

const updatedQuantums = await this.updatePriceLevel(

placeOrderResult,

perpetualMarket,

update

);

// WebSocket 메시지 생성 및 전송
```

```
if (this.shouldSendSubaccountMessage(orderPlace, placeOrderResult, placementStat

const subaccountMessage = this.createSubaccountWebsocketMessage(

redisOrder,

dbOrder,

perpetualMarket,

placementStatus

);

sendMessageWrapper(subaccountMessage, KafkaTopics.TO_WEBSOCKETS_SUBACCOUNTS);

}

}

}
```

주문 교체 로직

```
protected async updatePriceLevel(

result: PlaceOrderResult,

perpetualMarket: PerpetualMarketFromDatabase,

update: OffChainUpdateV1,

): Promise<number | undefined> {

if (result.replaced !== true || result.restingOnBook !== true) {

return undefined;

}


const remainingSizeDeltaInQuantums: Big = this.getRemainingSizeDeltaInQuantums(r
```

```
if (remainingSizeDeltaInQuantums.gt(0)) {
// 교체된 주문의 남은 수량만큼 주문서에서 제거
return OrderbookLevelsCache.updatePriceLevel({
ticker: perpetualMarket.ticker,
side: protocolTranslations.protocolOrderSideToOrderSide(result.oldOrder!.order!.
humanPrice: result.oldOrder!.price,
sizeDeltaInQuantums: remainingSizeDeltaInQuantums.mul(-1).toFixed(0),
client: redisClient,
});
}
}
```

2. 주문 업데이트 처리 (OrderUpdateHandler)

주문 업데이트는 주로 부분 체결 시 발생합니다:

```
export class OrderUpdateHandler extends Handler {
protected async handle(update: OffChainUpdateV1): Promise<void> {
const orderUpdate: OrderUpdateV1 = update.orderUpdate!;
// Redis에서 주문 업데이트
const updateResult: UpdateOrderResult = await updateOrder({
updatedOrderId: orderUpdate.orderId!,
newTotalFilledQuantums: orderUpdate.totalFilledQuantums.toNumber(),
client: redisClient,
```

```
});
if (updateResult.updated !== true) {
// Stateful 주문의 경우 캐시에 저장하여 나중에 재처리
if (isStatefulOrder(orderUpdate.orderId!.orderFlags)) {
await StatefulOrderUpdatesCache.addStatefulOrderUpdate(
OrderTable.orderIdToUuid(orderUpdate.orderId!),
orderUpdate,
Date.now(),
redisClient,
);
}
return;
}
// 완전 체결 여부 확인 및 오픈 주문 캐시 관리
const clobPairId: string = orderUpdate.orderId!.clobPairId.toString();
if (orderUpdate.totalFilledQuantums.gte(updateResult.order!.order!.quantums)) {
await OpenOrdersCache.removeOpenOrder(
updateResult.order!.id,
clobPairId,
redisClient,
);
} else {
await OpenOrdersCache.addOpenOrder(
```

```
updateResult.order!.id,
clobPairId,
redisClient,
);
}
// 주문서 레벨 업데이트
const sizeDeltaInQuantums: Big = this.getSizeDeltaInQuantums(updateResult, order
if (!sizeDeltaInQuantums.eq(0) &&
!requiresImmediateExecution(updateResult.order!.order!.timeInForce)) {
const updatedQuantums = await this.updatePriceLevel(updateResult, sizeDeltaInQua
// WebSocket 주문서 메시지 전송
const orderbookMessage = {
value: this.createOrderbookWebsocketMessage(
updateResult.order!,
perpetualMarket,
updatedQuantums,
),
};
sendMessageWrapper(orderbookMessage, KafkaTopics.TO_WEBSOCKETS_ORDERBOOKS);
}
}
}
```

가격 레벨 델타 계산

```
protected getSizeDeltaInQuantums(
updateResult: UpdateOrderResult,
orderUpdate: OrderUpdateV1,
): Big {
const cappedNewTotalFilledQuantums = this.getCappedNewTotalFilledQuantums(orderU
const cappedOldTotalFilledQuantums = this.getCappedOldTotalFilledQuantums(update
const orderQuantums = Big(updateResult.order!.order!.quantums.toNumber().toStrin

// 이전에 주문서에 없던 주문인 경우
if (updateResult.oldRestingOnBook === false) {
return orderQuantums.minus(cappedNewTotalFilledQuantums);
}

// 이전에 주문서에 있던 주문인 경우
return cappedOldTotalFilledQuantums.minus(cappedNewTotalFilledQuantums);
}
```

3. 주문 취소 처리 (OrderRemoveHandler)

주문 취소는 가장 복잡한 로직을 가집니다:

```
export class OrderRemoveHandler extends Handler {
```

```
protected async handle(update: OffChainUpdateV1): Promise<void> {

const orderRemove: OrderRemoveV1 = update.orderRemove!;

const reason: OrderRemovalReason = orderRemove.reason;

// Indexer 만료 메시지의 유효성 검증

if (reason === OrderRemovalReason.ORDER_REMOVAL_REASON_INDEXER_EXPIRED &&

!(await this.isOrderExpired(orderRemove))) {

return;

}

// Redis에서 주문 제거

const removeOrderResult: RemoveOrderResult = await removeOrder({

removedOrderId: orderRemove.removedOrderId!,

client: redisClient,

});

// 오픈 주문 캐시에서 제거

if (removeOrderResult.removed) {

const clobPairId: string = orderRemove.removedOrderId!.clobPairId.toString();

await OpenOrdersCache.removeOpenOrder(

removeOrderResult.removedOrder!.id,

clobPairId,

redisClient,

);

}

// Stateful 주문 취소와 일반 주문 제거 구분 처리
```

```
if (this.isStatefulOrderCancelation(orderRemove)) {
await this.handleStatefulOrderCancelation(orderRemove, removeOrderResult);
} else {
await this.handleOrderRemoval(orderRemove, removeOrderResult);
}
}
}
```

Stateful 주문 취소 처리

```
protected async handleStatefulOrderCancelation(
orderRemove: OrderRemoveV1,
removeOrderResult: RemoveOrderResult,
): Promise<void> {
// PostgreSQL에서 주문 정보 조회
const order: OrderFromDatabase | undefined = await OrderTable.findById(
OrderTable.orderIdToUuid(orderRemove.removedOrderId!)
);
// WebSocket 메시지 생성 및 전송
const subaccountMessage = {
value: this.createSubaccountWebsocketMessageFromPostgresOrder(
order,
orderRemove,
perpetualMarket.ticker,
```

```
),
};
sendMessageWrapper(subaccountMessage, KafkaTopics.TO_WEBSOCKETS_SUBACCOUNTS);
// 주문서에서 제거된 경우 주문서 레벨 업데이트
if (removeOrderResult.removed &&
removeOrderResult.restingOnBook === true &&
!requiresImmediateExecution(removeOrderResult.removedOrder!.order!.timeInForce))
await this.updateOrderbook(removeOrderResult, perpetualMarket);
}
}
```

데이터베이스 스키마 설계 및 인덱싱 전략

1. Redis 캐시 구조

Vulcan은 다음과 같은 Redis 캐시들을 관리합니다:

주요 캐시들:

- **OrdersCache**: 개별 주문 정보 저장
- **OrdersDataCache**: 주문의 체결 정보 저장
- **SubaccountOrderIdsCache**: 서브계정별 주문 ID 목록
- **OrderbookLevelsCache**: 가격 레벨별 주문 수량 집계
- **OpenOrdersCache**: 활성 주문 추적
- **CanceledOrdersCache**: 취소된 주문 추적

- **StatefulOrderUpdatesCache**: Stateful 주문 업데이트 임시 저장

캐시 업데이트 예시:

```
// 주문 생성 시 여러 캐시 동시 업데이트
const placeOrderResult: PlaceOrderResult = await placeOrder({
redisOrder,
client: redisClient,
});

// 가격 레벨 캐시 업데이트
await OrderbookLevelsCache.updatePriceLevel({
ticker: perpetualMarket.ticker,
side: protocolTranslations.protocolOrderSideToOrderSide(order.side),
humanPrice: redisOrder.price,
sizeDeltaInQuantums: remainingQuantums.toFixed(0),
client: redisClient,
});
```

2. PostgreSQL 연동

Vulcan은 특정 상황에서 PostgreSQL과 상호작용합니다:

Stateful 주문 처리:

```
// Stateful 주문 생성 시 DB에서 정보 조회
```

```
if (isStatefulOrder(redisOrder.order!.orderId!.orderFlags)) {
const orderUuid: string = OrderTable.orderIdToUuid(redisOrder.order!.orderId!);
dbOrder = await OrderTable.findById(orderUuid);
if (dbOrder === undefined) {
throw new Error(`Stateful order not found in database: ${orderUuid}`);
}
// 캐시된 주문 업데이트 재전송
await this.sendCachedOrderUpdate(orderUuid);
}
```

주문 상태 업데이트:

```
// 주문 취소 시 PostgreSQL 상태 업데이트
protected async cancelOrderInPostgres(
orderRemove: OrderRemoveV1,
): Promise<OrderFromDatabase | undefined> {
return OrderTable.update({
id: OrderTable.orderIdToUuid(orderRemove.removedOrderId!),
status: this.orderRemovalStatusToOrderStatus(orderRemove.removalStatus),
});
}
```

3. 인덱싱 최적화 전략

Redis 키 구조 최적화:

- 주문 ID 기반 해시 키 사용
- 서브계정별 주문 그룹화
- 가격 레벨별 효율적인 집계

데이터 일관성 보장:

- 원자적 Redis 연산 사용
- 트랜잭션 기반 PostgreSQL 업데이트
- 캐시 무효화 전략

이벤트 중복 처리 방지 메커니즘

1. 주문 만료 검증

Indexer에서 보낸 만료 메시지의 유효성을 검증합니다:

```
protected async isOrderExpired(orderRemove: OrderRemoveV1): Promise<boolean> {
// 최신 블록 정보 조회
const block: BlockFromDatabase = await BlockTable.getLatest({ readReplica: true
// Redis에서 주문 정보 조회
const redisOrder: RedisOrder | null = await OrdersCache.getOrder(
OrderTable.orderIdToUuid(orderRemove.removedOrderId!),
redisClient
);
if (redisOrder === null) {
```

```
return false; // 주문이 없으면 만료되지 않은 것으로 처리

}

const order: IndexerOrder = redisOrder.order!;

// 단기 주문의 goodTilBlock 검증

if (order.goodTilBlock! >= +block.blockHeight) {

return false; // 아직 만료되지 않음

}

return true;

}
```

2. Stateful 주문 업데이트 캐싱

Stateful 주문의 경우 온체인 이벤트 처리 전에 오프체인 업데이트가 도착할 수 있어 임시 캐싱을 사용합니다:

```
// 주문이 존재하지 않는 경우 캐시에 저장

if (updateResult.updated !== true) {

if (isStatefulOrder(orderFlags)) {

await StatefulOrderUpdatesCache.addStatefulOrderUpdate(

OrderTable.orderIdToUuid(orderUpdate.orderId!),

orderUpdate,

Date.now(),

redisClient,

);

}
```

```
return;
}

// 주문 생성 시 캐시된 업데이트 재전송
protected async sendCachedOrderUpdate(orderId: string): Promise<void> {
const cachedOrderUpdate: OrderUpdateV1 | undefined =
await StatefulOrderUpdatesCache.removeStatefulOrderUpdate(
orderId,
Date.now(),
redisClient,
);

if (cachedOrderUpdate !== undefined) {
const orderUpdateMessage: Message = {
key: getOrderIdHash(cachedOrderUpdate.orderId!),
value: Buffer.from(
Uint8Array.from(OffChainUpdateV1.encode({ orderUpdate: cachedOrderUpdate }).fini
),
};
sendMessageWrapper(orderUpdateMessage, KafkaTopics.TO_VULCAN);
}
}
```

3. 취소된 주문 추적

주문 취소 후 늦게 도착하는 체결 이벤트를 올바르게 처리하기 위해 취소된 주문을 추적합니다:

```
protected async addOrderToCanceledOrdersCache(
orderRemove: OrderRemoveV1,
timestampMs: number,
): Promise<void> {
const orderId: string = OrderTable.orderIdToUuid(orderRemove.removedOrderId!);


if (orderRemove.removalStatus ===
OrderRemoveV1_OrderRemovalStatus.ORDER_REMOVAL_STATUS_BEST_EFFORT_CANCELED) {
await CanceledOrdersCache.addBestEffortCanceledOrderId(orderId, timestampMs, red
} else if (orderRemove.removalStatus ===
OrderRemoveV1_OrderRemovalStatus.ORDER_REMOVAL_STATUS_CANCELED) {
await CanceledOrdersCache.addCanceledOrderId(orderId, timestampMs, redisClient);
}
}
```

WebSocket 메시지 큐잉 및 배치 전송

1. 메시지 큐잉 시스템

Vulcan은 효율적인 WebSocket 메시지 전송을 위해 큐잉 시스템을 사용합니다:

```
const queuedMessages: {[topic: string]: Message[]} = {};
const timeouts: {[topic: string]: NodeJS.Timeout } = {};

export function sendMessageWrapper(message: Message, topic: string): void {
if (queuedMessages[topic] === undefined) {
queuedMessages[topic] = [];
}
queuedMessages[topic].push(message);

// 큐 크기가 임계값에 도달하면 즉시 전송
if (shouldFlush(topic)) {
if (timeouts[topic] !== undefined) {
clearTimeout(timeouts[topic]!);
}
wrapBackgroundTask(sendMessages(topic), true, 'produce_websocket_messages');
}

// 타이머 기반 주기적 전송
if (timeouts[topic] === undefined) {
timeouts[topic] = setTimeout(() => {
wrapBackgroundTask(sendMessages(topic), true, 'produce_websocket_messages');
}, config.FLUSH_KAFKA_MESSAGES_INTERVAL_MS);
```

```
}

}


function shouldFlush(topic: string): boolean {

return (queuedMessages[topic]?.length ?? 0) > config.MAX_WEBSOCKET_MESSAGES_TO_Q

}
```

2. 배치 전송 및 에러 처리

```
async function sendMessages(topic: string): Promise<void> {

delete timeouts[topic];


const messages: Message[] = queuedMessages[topic];

if (messages === undefined || messages.length === 0) {

return;

}

queuedMessages[topic] = [];

if (!config.SEND_WEBSOCKET_MESSAGES) return;


let success: boolean = false;

try {

await producer.send({

topic,
```

```
messages,
});
success = true;
} catch (error) {
logger.error({
message: 'Failed to send messages to Kafka',
topic,
error,
numMessages: messages.length,
});


// 전송 실패 시 메시지 재큐잉
messages.forEach((message: Message) => sendMessageWrapper(message, topic));
} finally {
// 메트릭 수집
stats.histogram('vulcan.flush_websocket.size', messages.length, { topic, success
}
}
```

3. 메시지 타입별 처리

서브계정 메시지:

```
protected createSubaccountWebsocketMessage(
redisOrder: RedisOrder,
order: OrderFromDatabase | undefined,
perpetualMarket: PerpetualMarketFromDatabase,
placementStatus: OrderPlaceV1_OrderPlacementStatus,
): Buffer {
const contents: SubaccountMessageContents = {
orders: [{
id: OrderTable.orderIdToUuid(redisOrder.order!.orderId!),
subaccountId: SubaccountTable.subaccountIdToUuid(redisOrder.order!.orderId!.suba
clientId: redisOrder.order!.orderId!.clientId.toString(),
clobPairId: perpetualMarket.clobPairId,
side: protocolTranslations.protocolOrderSideToOrderSide(redisOrder.order!.side),
size: redisOrder.size,
price: redisOrder.price,
status: placementStatus === OrderPlaceV1_OrderPlacementStatus.ORDER_PLACEMENT_ST
? APIOrderStatusEnum.OPEN
: APIOrderStatusEnum.BEST_EFFORT_OPENED,
// ... 기타 필드들
}],
};
```

```
const subaccountMessage: SubaccountMessage = SubaccountMessage.fromPartial({

contents: JSON.stringify(contents),

subaccountId: redisOrder.order!.orderId!.subaccountId!,

version: SUBACCOUNTS_WEBSOCKET_MESSAGE_VERSION,

});



return Buffer.from(Uint8Array.from(SubaccountMessage.encode(subaccountMessage).f

}
```

주문서 메시지:

```
protected createOrderbookWebsocketMessage(

redisOrder: RedisOrder,

perpetualMarket: PerpetualMarketFromDatabase,

updatedQuantums: number,

): Buffer {

const orderbookSide: OrderbookSide = orderSideToOrderbookSide(redisOrder.order!.

const humanSize: string = protocolTranslations.quantumsToHumanFixedString(

updatedQuantums.toString(),

perpetualMarket.atomicResolution,

);

const contents: OrderbookMessageContents = {

[orderbookSide]: [[redisOrder.price, humanSize]],
```

```
};

const orderbookMessage: OrderbookMessage = OrderbookMessage.fromPartial({
clobPairId: perpetualMarket.clobPairId,
contents: JSON.stringify(contents),
version: ORDERBOOKS_WEBSOCKET_MESSAGE_VERSION,
});

return Buffer.from(Uint8Array.from(OrderbookMessage.encode(orderbookMessage).fin
}
```

성능 최적화 및 모니터링

1. 메트릭 수집

```
// 메시지 처리 시간 측정
stats.timing(
`${config.SERVICE_NAME}.processed_update.timing`,
Date.now() - start,
STATS_NO_SAMPLING,
{
success: success.toString(),
messageType: getMessageType(update),
```

```
}

);


// 배치 처리 메트릭

stats.timing('vulcan.batch_processing_time', batchProcessingTime, metricTags);

stats.timing('vulcan.batch_size', batch.messages.length, metricTags);
```

2. 에러 처리 및 복구

```
try {

await handler.handleUpdate(update);

success = true;

} catch (error) {

if (error instanceof ParseMessageError) {

// 파싱 에러는 재시도하지 않음

logger.crit({

message: 'Unable to parse message, this must be due to a bug in the V4 node',

error,

});

} else {

// 기타 에러는 재시도를 위해 에러 전파

logger.error({

message: 'Unable to process message',
```

```
error,

});

throw error;

}

}
```

3. 메모리 및 연결 관리

```
// 서비스 종료 시 정리 작업

process.on('SIGTERM', async () => {

await stopConsumer();

await flushAllQueues(); // 큐에 남은 메시지 모두 전송

redisClient.quit();

process.exit(0);

});
```

결론

Vulcan 서비스는 dYdX v4의 오프체인 주문 처리 파이프라인의 핵심으로, 실시간 주문서 관리, 효율적인 캐싱, 그리고 안정적인 WebSocket 메시징을 통합한 고성능 아키텍처를 제공합니다. 복잡한 주문 상태 관리, 중복 처리 방지, 그리고 확장 가능한 메시지 큐잉 시스템을 통해 높은 처리량과 데이터 일관성을 달성하고 있습니다.